

Лабораторная работа 5: Добавление работы с базой данных в приложение

Цель работы

Добавить в приложение хранение данных в реляционной базе данных с использованием Entity Framework Core, обеспечив сохранность данных между запусками приложения.

Задачи

5.1 Выбор и установка СУБД

Для работы приложения необходима установленная реляционная СУБД. Выберите один из вариантов:

Вариант 1: SQL Server в колледже (при работе на компьютерах ВКИ)

В колледже развернут SQL Server во внутренней сети с доменной аутентификацией.

Строка подключения:

```
Server=<имя_сервера>;Database=<ваше_имя>_<название_проекта>;Integrated Security=True;
```

Пример:

```
Server=SQL-EDU;Database=IvanovI_BeautyDB;Integrated Security=True;
```

Примечание: Уточните имя сервера у преподавателя. Аутентификация происходит автоматически через доменный аккаунт.

Вариант 2: SQL Server LocalDB (для личного компьютера)

Упрощённая версия SQL Server для разработки, запускается автоматически при первом подключении.

Проверка наличия:

```
sqllocaldb info
```

Установка через Visual Studio Installer:

1. Запустите Visual Studio Installer
2. Выберите **Modify** → **Individual components**
3. Отметьте **SQL Server Express LocalDB**
4. Нажмите **Modify**

Запуск:

```
sqllocaldb start MSSQLLocalDB  
sqllocaldb info MSSQLLocalDB
```

Строка подключения:

```
Server=(localdb)\MSSQLLocalDB;Database=<ИмяБД>;Integrated Security=True;
```

Вариант 3: SQL Server Express

Полнофункциональная бесплатная версия SQL Server (ограничение на размер БД: 10 GB).

Установка:

1. Скачайте SQL Server Express: <https://www.microsoft.com/sql-server/sql-server-downloads>
2. Выберите **Basic** установку
3. Следуйте инструкциям установщика

Строка подключения:

```
Server=localhost\SQLEXPRESS;Database=<ИмяБД>;Trusted_Connection=True;
```

Вариант 4: MySQL

Open-source СУБД, кроссплатформенная альтернатива SQL Server.

Установка:

1. Скачайте MySQL Community Server: <https://dev.mysql.com/downloads/mysql/>
2. Или установите MySQL Installer (включает Workbench): <https://dev.mysql.com/downloads/installer/>
3. Задайте пароль для пользователя `root` во время установки

Строка подключения:

```
Server=localhost;Database=<ИмяБД>;User=root;Password=<ВашПароль>;
```

Важно: Для MySQL потребуются другие NuGet-пакеты (см. шаг 5.3).

GUI-инструменты (опционально)

Для SQL Server:

- SQL Server Management Studio (SSMS): <https://learn.microsoft.com/sql/ssms/download-sql-server-management-studio-ssms>
- Azure Data Studio: <https://learn.microsoft.com/azure-data-studio/download-azure-data-studio>

Для MySQL:

- MySQL Workbench: <https://dev.mysql.com/downloads/workbench/>

Примечание: Entity Framework Core создаст базу данных автоматически, GUI-инструмент нужен только для просмотра и отладки.

Введение в Entity Framework Core

Entity Framework Core (EF Core) — это современный ORM (Object-Relational Mapper) от Microsoft для .NET приложений. ORM — это технология, которая позволяет работать с базой данных через объекты C# вместо написания SQL-запросов.

Зачем нужен EF Core?

Без ORM (используя ADO.NET напрямую):

```
using var connection = new SqlConnection(connectionString);
connection.Open();
var command = new SqlCommand(
    "INSERT INTO Reservations (ClientName, Date, Status) VALUES (@name, @date, @status)",
    connection);
command.Parameters.AddWithValue("@name", reservation.ClientName);
command.Parameters.AddWithValue("@date", reservation.Date);
command.Parameters.AddWithValue("@status", reservation.Status);
command.ExecuteNonQuery();
```

C EF Core:

```
_context.Reservations.Add(reservation);
_context.SaveChanges();
```

Основные преимущества EF Core

1. **Меньше кода:** Не нужно писать SQL вручную для базовых операций
2. **Типобезопасность:** Ошибки обнаруживаются на этапе компиляции, а не выполнения
3. **Автоматическое отслеживание изменений:** EF Core сам понимает, какие объекты изменились
4. **Поддержка разных СУБД:** Один код работает с SQL Server, MySQL, PostgreSQL, SQLite
5. **Миграции:** Автоматическое создание и обновление схемы базы данных

Ключевые концепции EF Core

1. DbContext

DbContext — это центральный класс EF Core, представляющий сессию работы с базой данных.

```
public class AppDbContext : DbContext
{
    public DbSet<Reservation> Reservations { get; set; }
    public DbSet<Master> Masters { get; set; }
}
```

Ключевые задачи DbContext:

- Управляет подключением к базе данных
- Отслеживает изменения объектов
- Выполняет запросы к БД
- Сохраняет изменения (SaveChanges())

2. DbSet

DbSet<T> представляет таблицу в базе данных. Каждое свойство типа DbSet<T> в DbContext соответствует одной таблице.

```
// Добавление записи
context.Reservations.Add(newReservation);

// Получение всех записей
var all = context.Reservations.ToList();

// Поиск по ID
var reservation = context.Reservations.Find(id);

// Фильтрация с помощью LINQ
```

```
var today = DateOnly.FromDateTime(DateTime.Now);
var filtered = context.Reservations
    .Where(r => r.Date > today)
    .ToList();
```

3. Сущности (Entities)

Сущности — это обычные C# классы, которые EF Core сопоставляет с таблицами БД.

```
public class Reservation
{
    public int Id { get; set; } // Первичный ключ
    public string ClientName { get; set; } // Колонка ClientName
    public DateOnly Date { get; set; } // Колонка Date
    public ReservationStatus Status { get; set; }

    // Навигационное свойство (связь с другой таблицей)
    public Master Master { get; set; }
    public int MasterId { get; set; } // Внешний ключ
}
```

Важно:

- Свойство Id или <ИмяКласса>Id автоматически становится первичным ключом
- Свойства типа другой сущности автоматически создают связь (foreign key)

4. Миграции (Migrations)

Миграции — это способ версионирования схемы базы данных. EF Core генерирует код на C#, который создаёт или обновляет таблицы.

Создание миграции:

```
dotnet ef migrations add InitialCreate
```

Применение миграции к БД:

```
dotnet ef database update
```

После выполнения этих команд EF Core:

1. Анализирует классы сущностей
2. Генерирует SQL-скрипты для создания таблиц
3. Создаёт таблицы в базе данных

Какие запросы генерирует EF Core?

1. Запрос данных:

```
var reservations = context.Reservations
    .Where(r => r.Status == ReservationStatus.Confirmed)
    .ToList();
```

EF Core преобразует LINQ в SQL:

```
SELECT * FROM Reservations WHERE Status = 1
```

2. Отслеживание изменений:

```
var reservation = context.Reservations.Find(1);
reservation.Status = ReservationStatus.Completed; // Объект помечен как изменённый
context.SaveChanges(); // Генерируется UPDATE запрос
```

Сгенерированный SQL:

```
UPDATE Reservations SET Status = 2 WHERE Id = 1
```

3. Добавление новой записи:

```
context.Reservations.Add(newReservation);
context.SaveChanges();
```

Сгенерированный SQL:

```
INSERT INTO Reservations (ClientName, Date, Status) VALUES (...)
```

Создание проекта Data.SqlServer (или Data.MySQL)

Шаг 1: Создание Class Library проекта

1. В Visual Studio откройте Solution Explorer
2. Щелкните правой кнопкой на Solution → **Add** → **New Project**
3. Выберите шаблон **Class Library** (не .NET Framework!)
4. Назовите проект:
 - Для SQL Server: <ИмяПроекта>.Data.SqlServer (например, BeautyDB.Data.SqlServer)
 - Для MySQL: <ИмяПроекта>.Data.MySQL (например, BeautyDB.Data.MySQL)
5. Выберите ту же версию .NET, что и в других проектах (например, .NET 8.0)
6. Нажмите **Create**

Шаг 2: Добавление ссылок между проектами

Проект Data.SqlServer (или Data.MySQL) должен знать о:

- **Domain** — чтобы работать с сущностями (Reservation, Master и т.д.)
- **Data.Interfaces** — чтобы реализовывать интерфейсы репозитория

Добавить ссылки:

1. Щелкните правой кнопкой на <ИмяПроекта>.Data.SqlServer (или Data.MySQL) → **Add** → **Project Reference**
2. Отметьте галочками:
 - <ИмяПроекта>.Domain
 - <ИмяПроекта>.Data.Interfaces
3. Нажмите **ОК**

Шаг 3: Обновление ссылок в UI проекте

Теперь UI проект должен знать о новом проекте:

1. Щелкните правой кнопкой на <ИмяПроекта>.UI → **Add** → **Project Reference**
2. Отметьте галочкой:
 - <ИмяПроекта>.Data.SqlServer (или Data.MySQL)
3. Нажмите **ОК**

Примечание: Ссылку на `Data.InMemory` можно оставить для возможности переключения между хранилищами.

Ожидаемая структура проекта после создания

```
Solution/
├── <ИмяПроекта>.UI/           # WPF формы и элементы управления
├── <ИмяПроекта>.Domain/       # Классы предметной области
├── <ИмяПроекта>.Services/     # Бизнес-логика и сервисы
├── <ИмяПроекта>.Data.Interfaces/ # Интерфейсы репозитория
├── <ИмяПроекта>.Data.InMemory/ # In-memory реализация хранения данных
└── <ИмяПроекта>.Data.SqlServer/ # EF Core для SQL Server (Новое)
    (или <ИмяПроекта>.Data.MySQL) # EF Core для MySQL (альтернатива)
```

Установка NuGet-пакетов Entity Framework Core

Entity Framework Core распространяется через NuGet-пакеты. Необходимо установить 2 пакета: провайдер для СУБД и пакет Design.

Важно: Все пакеты EF Core должны быть одной версии и соответствовать версии .NET:

- Для .NET 8 используйте версии EF Core 8.x (например, 8.0.11)
- Для .NET 9 используйте версии EF Core 9.x (например, 9.0.0)

Microsoft.EntityFrameworkCore.SqlServer (для SQL Server)

Зачем нужен: Провайдер для подключения к SQL Server. Содержит драйверы и адаптеры для преобразования LINQ-запросов в SQL-запросы для SQL Server.

Куда устанавливать: `<ИмяПроекта>.Data.SqlServer`

Когда устанавливать: Если вы выбрали SQL Server, LocalDB или SQL Server Express на шаге 5.1.

Установка:

1. В Solution Explorer щелкните правой кнопкой на проекте `<ИмяПроекта>.Data.SqlServer`
2. Выберите **Manage NuGet Packages**
3. Перейдите на вкладку **Browse**
4. В поле поиска введите: `Microsoft.EntityFrameworkCore.SqlServer`
5. Выберите пакет из списка
6. Выберите версию (например, 8.0.11 для .NET 8 или 9.0.0 для .NET 9)
7. Нажмите **Install**
8. Подтвердите установку, нажав **OK** в диалоговом окне

Pomelo.EntityFrameworkCore.MySql (для MySQL)

Зачем нужен: Провайдер для подключения к MySQL. Это альтернатива пакету SqlServer для тех, кто выбрал MySQL на шаге 5.1.

Куда устанавливать: `<ИмяПроекта>.Data.MySQL`

Когда устанавливать: Если вы выбрали MySQL на шаге 5.1. **Не устанавливайте вместе с SqlServer** - нужен только один провайдер.

Установка:

1. В Solution Explorer щелкните правой кнопкой на проекте `<ИмяПроекта>.Data.MySQL`

2. Выберите **Manage NuGet Packages**
3. Перейдите на вкладку **Browse**
4. В поле поиска введите: `Pomelo.EntityFrameworkCore.MySql`
5. Выберите пакет из списка
6. Выберите версию (например, 8.0.2 для .NET 8 или 9.0.0 для .NET 9)
7. Нажмите **Install**

Microsoft.EntityFrameworkCore.Design

Зачем нужен: Инструменты для генерации миграций. Позволяет создавать код миграций на основе изменений в классах-сущностях.

Куда устанавливать: `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`)

Когда устанавливать: Всегда (для любой СУБД).

Установка:

1. В том же окне **Manage NuGet Packages** для вашего проекта (`Data.SqlServer` или `Data.MySQL`)
2. В поле поиска введите: `Microsoft.EntityFrameworkCore.Design`
3. Выберите пакет
4. Выберите ту же версию, что и для провайдера (например, 8.0.11 для .NET 8 или 9.0.0 для .NET 9)
5. Нажмите **Install**

Проверка установки

После установки всех пакетов:

1. Закройте окна **NuGet Package Manager**
2. Соберите решение: **Build** → **Build Solution** (`Ctrl+Shift+B`)
3. Убедитесь, что сборка прошла успешно без ошибок

Создание DbContext

`DbContext` — это класс, который управляет подключением к базе данных и позволяет работать с таблицами через свойства `DbSet<T>` .

Создание класса AppDbContext

В проекте `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`) создайте файл `AppDbContext.cs` (или `<ИмяПроекта>DbContext.cs`).

Шаблон:

```
using Microsoft.EntityFrameworkCore;
using <ИмяПроекта>.Domain;

namespace <ИмяПроекта>.Data.SqlServer;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }
}

// Добавьте DbSet для каждой сущности из Domain
public DbSet<<Сущность1>> <Сущность1> { get; set; }
```

```
public DbSet<<Сущность2>> <Сущности2> { get; set; }  
}
```

Конкретный пример для салона красоты:

```
using Microsoft.EntityFrameworkCore;  
using BeautyDB.Domain;  
  
namespace BeautyDB.Data.SqlServer;  
  
public class BeautyDbContext : DbContext  
{  
    public BeautyDbContext(DbContextOptions<BeautyDbContext> options) : base(options)  
    {  
    }  
  
    public DbSet<Reservation> Reservations { get; set; }  
    public DbSet<Master> Masters { get; set; }  
    public DbSet<Client> Clients { get; set; }  
    public DbSet<Service> Services { get; set; }  
}
```

Пояснение

Конструктор с DbContextOptions:

```
public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
```

- Принимает настройки подключения (строка подключения, провайдер БД)
- Передает их базовому классу `DbContext`
- Позволяет настраивать подключение извне при создании контекста

Свойства DbSet:

```
public DbSet<Reservation> Reservations { get; set; }
```

- Каждое свойство `DbSet<T>` представляет таблицу в БД
- Имя свойства обычно совпадает с именем таблицы (во множественном числе)
- Добавьте `DbSet` для каждой таблицы. В качестве типа сущности укажите классы из проекта `Domain`.

Настройка модели (OnModelCreating)

Для сложных сценариев используйте метод `OnModelCreating` для настройки отношений между сущностями и параметров хранения.

Owned Entities (вложенные объекты без отдельной таблицы)

Owned Entities — это объекты, которые не имеют собственного жизненного цикла и всегда существуют только в контексте родительской сущности. EF Core хранит их поля как колонки родительской таблицы, а не создаёт отдельную таблицу.

Когда использовать:

- Адреса (Address)
- Контактные данные (Client в Reservation)

- Объекты без собственного Id
- Данные, которые всегда привязаны к родителю

Пример:

Если в вашем проекте Client всегда существует только в контексте бронирования:

```
public class BeautyDbContext : DbContext
{
    public BeautyDbContext(DbContextOptions<BeautyDbContext> options) : base(options)
    {
    }

    public DbSet<Reservation> Reservations { get; set; }
    public DbSet<Master> Masters { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // Client хранится в таблице Reservations
        modelBuilder.Entity<Reservation>()
            .OwnsOne(r => r.Client);
    }
}
```

Результат в базе данных:

Вместо отдельной таблицы Clients, поля Client будут храниться прямо в таблице Reservations:

```
Reservations
├─ Id (Guid)
├─ Client_Name (nvarchar)    ← Поле из Client
├─ Client_Phone (nvarchar)  ← Поле из Client
├─ MasterId (Guid)
├─ ServiceDescription (nvarchar)
└─ ...
```

Альтернатива без Owned Entity:

Если Client должен быть независимой сущностью (с возможностью переиспользования между бронированиями):

```
// Создайте DbSet<Client>
public DbSet<Client> Clients { get; set; }

// Client будет храниться в отдельной таблице с собственным Id
```

Настройка nullable отношений

EF Core автоматически определяет nullable отношения по типу свойства (Master? vs Master), но для явного контроля можно настроить их вручную.

Пример для nullable FK:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
```

```
// Явная настройка связи Reservation -> Master
modelBuilder.Entity<Reservation>()
    .HasOne(r => r.Master)           // У Reservation есть один Master
    .WithMany()                     // У Master много Reservations (не храним обратную
навигацию)
    .HasForeignKey("MasterId")      // Внешний ключ называется MasterId
    .IsRequired(false);             // Master необязателен (NULL разрешен)
}
```

Результат в базе данных:

```
CREATE TABLE Reservations (
    Id uniqueidentifier PRIMARY KEY,
    MasterId uniqueidentifier NULL, -- Может быть NULL
    FOREIGN KEY (MasterId) REFERENCES Masters(Id)
);
```

Когда использовать явную настройку:

- Нестандартные названия внешних ключей
- Каскадное удаление (`.OnDelete(DeleteBehavior.Cascade)`)
- Сложные связи many-to-many
- Композитные ключи

Примечание: Для простых случаев EF Core определит nullable автоматически из `Master?` в классе, и явная настройка не обязательна.

Миграции Entity Framework Core

Миграции — это механизм версионирования схемы базы данных. EF Core анализирует классы-сущности и автоматически генерирует C# код, который создаёт или обновляет таблицы в базе данных.

Зачем нужны миграции?

Проблема без миграций:

- При изменении классов нужно вручную писать SQL-скрипты для обновления БД
- Легко допустить ошибку и потерять данные
- Сложно синхронизировать схему БД между разработчиками

Решение с миграциями:

- EF Core автоматически создаёт SQL-скрипты на основе изменений в классах
- Миграции можно версионировать в Git
- Легко откатиться к предыдущей версии схемы БД
- Схема БД синхронизируется автоматически при запуске приложения

Установка инструментов миграций

Для работы с миграциями необходимо установить глобальный инструмент `dotnet-ef`.

Проверка наличия:

```
dotnet ef --version
```

Если инструмент не установлен:

Версия `dotnet-ef` должна соответствовать версии EF Core пакетов в проекте.

Для .NET 8 (EF Core 8.x):

```
dotnet tool install --global dotnet-ef --version 8.0.11
```

Для .NET 9 (EF Core 9.x):

```
dotnet tool install --global dotnet-ef --version 9.0.0
```

Обновление до последней версии (если уже установлен):

```
dotnet tool update --global dotnet-ef
```

Переустановка на конкретную версию:

```
dotnet tool uninstall --global dotnet-ef  
dotnet tool install --global dotnet-ef --version 8.0.11
```

Создание первой миграции

Миграция — это снимок текущего состояния классов-сущностей. Первая миграция обычно называется `InitialCreate`.

Для создания миграции откройте терминал, перейдите в корень решения и выполните команду:

```
dotnet ef migrations add InitialCreate --project <ИмяПроекта>\<ИмяПроекта>.Data.SqlServer
```

Или для MySQL:

```
dotnet ef migrations add InitialCreate --project <ИмяПроекта>\<ИмяПроекта>.Data.MySQL
```

Что происходит при выполнении команды:

1. EF Core анализирует все классы, указанные в `DbSet<T>` в вашем `DbContext`
2. Генерирует файлы миграции в папке `Migrations/`:
 - `<Timestamp>_InitialCreate.cs` — код миграции (создание таблиц)
 - `<Timestamp>_InitialCreate.Designer.cs` — метаданные миграции
 - `<ИмяПроекта>DbContextModelSnapshot.cs` — снимок текущей модели

Пример содержимого файла миграции (фрагмент):

```
public partial class InitialCreate : Migration  
{  
    protected override void Up(MigrationBuilder migrationBuilder)  
    {  
        migrationBuilder.CreateTable(  
            name: "Reservations",  
            columns: table => new  
            {  
                Id = table.Column<Guid>(nullable: false),  
                ClientName = table.Column<string>(nullable: false),  
                Date = table.Column<DateOnly>(nullable: false),  
                Status = table.Column<int>(nullable: false)  
            },  
            constraints: table =>
```

```

        {
            table.PrimaryKey("PK_Reservations", x => x.Id);
        });
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(name: "Reservations");
    }
}

```

Методы миграции:

- `Up()` — применяет миграцию (создаёт таблицы)
- `Down()` — откатывает миграцию (удаляет таблицы)

Применение миграции к базе данных

После создания миграции её нужно применить к базе данных. Это можно сделать двумя способами:

Способ 1: Применение через команду (для тестирования)

```
dotnet ef database update --project <ИмяПроекта>\<ИмяПроекта>.Data.SqlServer
```

Конкретный пример:

```
dotnet ef database update --project BeautyDB\BeautyDB.Data.SqlServer
```

Что происходит:

1. EF Core подключается к базе данных (используя строку подключения из конфигурации)
2. Создаёт БД, если её не существует
3. Создаёт служебную таблицу `__EFMigrationsHistory` для отслеживания применённых миграций
4. Выполняет метод `Up()` всех неприменённых миграций
5. Создаёт таблицы и столбцы согласно вашим классам-сущностям

Примечание: Для этого способа необходимо настроить строку подключения в фабрике `DbContext` (см. следующий раздел). Альтернативно, можно применять миграции автоматически при запуске приложения (способ 2).

Способ 2: Применение автоматически при запуске приложения (рекомендуется)

Добавьте вызов `Database.Migrate()` в код инициализации приложения. Это автоматически применит все неприменённые миграции при запуске.

Пример (будет подробно рассмотрен в разделе "Замена InMemory на SqlServer в UI"):

```

// В Main Window или App.xaml.cs
var context = new AppDbContext(options);
context.Database.Migrate(); // Автоматически применяет миграции

```

Преимущества этого способа:

- Миграции применяются автоматически при первом запуске
- Не нужно помнить про команду `database update`
- Работает на всех компьютерах (коллег, преподавателя) без дополнительных действий

Проверка результата

После применения миграции проверьте, что таблицы созданы:

Для SQL Server / LocalDB (через SSMS):

1. Скачайте и установите SQL Server Management Studio (SSMS): <https://learn.microsoft.com/sql/ssms/download-sql-server-management-studio-ssms>
2. Подключитесь к серверу (например, (localdb)\MSSQLLocalDB или localhost\SQLEXPRESS)
3. Разверните **Databases** → **<ИмяВашейБД>** → **Tables**
4. Вы должны увидеть таблицы, соответствующие вашим сущностям, и служебную таблицу `dbo.__EFMigrationsHistory`

Для MySQL (через MySQL Workbench):

1. Откройте MySQL Workbench
2. Подключитесь к серверу
3. Разверните вашу БД в списке схем
4. Проверьте наличие таблиц

Добавление новых миграций при изменении классов

При каждом изменении классов-сущностей (добавление/удаление полей, изменение типов) нужно создавать новую миграцию.

Пример изменения:

Допустим, вы добавили поле `Email` в класс `Client`:

```
public class Client
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; } // Новое поле
}
```

Создание миграции:

```
dotnet ef migrations add AddEmailToClient --project <ИмяПроекта>\<ИмяПроекта>.Data.SqlServer
```

Применение миграции:

```
dotnet ef database update --project <ИмяПроекта>\<ИмяПроекта>.Data.SqlServer
```

Или автоматически при следующем запуске приложения (если используете `context.Database.Migrate()`).

Пример сгенерированной миграции:

```
public partial class AddEmailToClient : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<string>(
            name: "Email",
            table: "Clients",
            type: "nvarchar(max)",
            nullable: false,
        );
    }
}
```

```

        defaultValue: "");
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "Email",
            table: "Clients");
    }
}

```

Обратите внимание:

- EF Core автоматически определил, что нужно добавить столбец `Email` в таблицу `Clients`
- Метод `Up()` добавляет столбец
- Метод `Down()` удаляет столбец (для отката миграции)
- EF Core сам определил тип данных SQL (`nvarchar(max)` для `string`)

Настройка строки подключения (Connection String)

Строка подключения содержит информацию о том, как приложение должно подключиться к базе данных: адрес сервера, имя БД, способ аутентификации.

Мы будем хранить строку подключения в файле `appsettings.database.json`, который будет использоваться как при создании миграций, так и при работе приложения.

Создание `appsettings.database.json`

Шаг 1: Создать файл

1. В проекте `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`) щелкните правой кнопкой → **Add** → **New Item**
2. Выберите **JSON File**
3. Назовите файл `appsettings.database.json`
4. Нажмите **Add**

Шаг 2: Заполнить содержимое

Для SQL Server:

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=<ИмяБД>;Integrated
Security=True;TrustServerCertificate=True;"
  }
}

```

Для MySQL:

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=<ИмяБД>;User=root;Password=<ВашПароль>;"
  }
}

```

Шаг 3: Настроить копирование файла при сборке

1. В Solution Explorer найдите файл `appsettings.database.json`
2. Щелкните на нём правой кнопкой → **Properties**
3. Установите **Copy to Output Directory** → **Copy if newer**

Установка NuGet-пакетов для чтения конфигурации

В проект `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`) установите пакеты:

1. Щелкните правой кнопкой → **Manage NuGet Packages**
2. Установите (той же версии, что и EF Core):
 - `Microsoft.Extensions.Configuration`
 - `Microsoft.Extensions.Configuration.Json`

Создание фабрики для создания DbContext

Фабрика позволяет создавать экземпляры `DbContext` как для команд `dotnet ef migrations`, так и для использования в UI проекте.

В проекте `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`) создайте файл `<ИмяПроекта>DbContextFactory.cs`:

Шаблон:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;
using Microsoft.Extensions.Configuration;

namespace <ИмяПроекта>.Data.SqlServer;

public class <ИмяПроекта>DbContextFactory : IDesignTimeDbContextFactory<<ИмяПроекта>DbContext>
{
    public <ИмяПроекта>DbContext CreateDbContext(string[] args)
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.database.json")
            .Build();

        return CreateDbContext(configuration);
    }

    public <ИмяПроекта>DbContext CreateDbContext(IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString("DefaultConnection");

        var optionsBuilder = new DbContextOptionsBuilder<<ИмяПроекта>DbContext>();
        optionsBuilder.UseSqlServer(connectionString);

        return new <ИмяПроекта>DbContext(optionsBuilder.Options);
    }
}
```

Конкретный пример для BeautyDB (SQL Server):

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;
using Microsoft.Extensions.Configuration;

namespace BeautyDB.Data.SqlServer;

public class BeautyDbContextFactory : IDesignTimeDbContextFactory<BeautyDbContext>
```

```

{
    public BeautyDbContext CreateDbContext(string[] args)
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.database.json")
            .Build();

        return CreateDbContext(configuration);
    }

    public BeautyDbContext CreateDbContext(IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString("DefaultConnection");

        var optionsBuilder = new DbContextOptionsBuilder<BeautyDbContext>();
        optionsBuilder.UseSqlServer(connectionString);

        return new BeautyDbContext(optionsBuilder.Options);
    }
}

```

Для MySQL замените строку:

```
optionsBuilder.UseMySQL(connectionString, ServerVersion.AutoDetect(connectionString));
```

Пояснение:

- Метод `CreateDbContext(string[] args)` используется командами `dotnet ef` — читает конфигурацию из файла
- Метод `CreateDbContext(IConfiguration configuration)` будет использоваться в UI — принимает готовую конфигурацию
- Оба метода создают одинаковый `DbContext` с одной строкой подключения

Проверка работы

Соберите решение и создайте первую миграцию:

```
dotnet ef migrations add InitialCreate --project <ИмяПроекта>\<ИмяПроекта>.Data.SqlServer
```

Если миграция создалась успешно — всё настроено правильно. Файлы миграции появятся в папке `Migrations/` вашего проекта `Data.SqlServer` (или `Data.MySQL`).

Реализация репозиторий с использованием EF Core

Теперь нужно реализовать интерфейсы репозиторий из `Data.Interfaces`, используя EF Core вместо `List<T>`.

В проекте `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`) создайте класс `<Сущность>Repository.cs` для каждой сущности, с которой вы работали в `InMemory`.

Шаблон:

```

using <ИмяПроекта>.Data.Interfaces;
using <ИмяПроекта>.Domain;

```



```

namespace <ИмяПроекта>.Data.SqlServer;

public class <Сущность>Repository : I<Сущность>Repository
{
    private readonly <ИмяПроекта>DbContext _context;

    // Конструктор принимает DbContext через dependency injection
    public <Сущность>Repository(<ИмяПроекта>DbContext context)
    {
        _context = context;
    }

    public int Add(<Сущность> entity)
    {
        // Add() добавляет сущность в отслеживание EF Core
        _context.<Сущности>.Add(entity);

        // SaveChanges() генерирует и выполняет SQL INSERT
        // EF Core автоматически присваивает Id после сохранения
        _context.SaveChanges();

        return entity.Id;
    }

    public <Сущность>? GetById(int id)
    {
        // Find() ищет сущность по первичному ключу
        // Сначала проверяет кэш, затем выполняет SELECT
        // Возвращает null, если не найдена
        return _context.<Сущности>.Find(id);
    }

    public List<<Сущность>> GetAll(<Сущность>Filter filter)
    {
        // AsQueryable() создаёт LINQ-запрос (ещё не выполнен)
        var query = _context.<Сущности>.AsQueryable();

        // ВАЖНО: Если сущность имеет навигационные свойства (связи),
        // используйте Include() для их загрузки:
        // var query = _context.<Сущности>
        //     .Include(x => x.Master) // Загружаем связанную сущность
        //     .AsQueryable();

        // Добавляем фильтры – формируется SQL WHERE
        if (filter.StartDate.HasValue)
            query = query.Where(x => x.Date >= filter.StartDate.Value);

        if (filter.EndDate.HasValue)
            query = query.Where(x => x.Date <= filter.EndDate.Value);

        // ToList() выполняет запрос к БД и возвращает результат
        return query.ToList();
    }

    public bool Update(<Сущность> entity)
    {
        // Сначала находим существующую сущность
        var existing = _context.<Сущности>.Find(entity.Id);
        if (existing == null)
            return false;

        // Копируем все свойства (кроме Id) из entity в existing

```

```

        existing.CopyFrom(entity);

        // SaveChanges() генерирует SQL UPDATE только для изменённых полей
        _context.SaveChanges();
        return true;
    }

    public bool Delete(int id)
    {
        var entity = _context.<Сущности>.Find(id);
        if (entity == null)
            return false;

        // Remove() помечает сущность для удаления
        _context.<Сущности>.Remove(entity);

        // SaveChanges() генерирует и выполняет SQL DELETE
        _context.SaveChanges();
        return true;
    }
}

```

Конкретный пример для BeautyDB (Reservation):

```

using BeautyDB.Data.Interfaces;
using BeautyDB.Domain;

namespace BeautyDB.Data.SqlServer;

public class ReservationRepository : IReservationRepository
{
    private readonly BeautyDbContext _context;

    public ReservationRepository(BeautyDbContext context)
    {
        _context = context;
    }

    public int Add(Reservation reservation)
    {
        _context.Reservations.Add(reservation);
        _context.SaveChanges();
        return reservation.Id;
    }

    public Reservation? GetById(int id)
    {
        return _context.Reservations.Find(id);
    }

    public List<Reservation> GetAll(ReservationsFilter filter)
    {
        var query = _context.Reservations.AsQueryable();

        if (filter.StartDate.HasValue)
            query = query.Where(r => r.Date >= filter.StartDate.Value);

        if (filter.EndDate.HasValue)
            query = query.Where(r => r.Date <= filter.EndDate.Value);

        return query.ToList();
    }
}

```

```

    }

    public bool Update(Reservation reservation)
    {
        var existing = _context.Reservations.Find(reservation.Id);
        if (existing == null)
            return false;

        existing.CopyFrom(reservation);
        _context.SaveChanges();
        return true;
    }

    public bool Delete(int id)
    {
        var reservation = _context.Reservations.Find(id);
        if (reservation == null)
            return false;

        _context.Reservations.Remove(reservation);
        _context.SaveChanges();
        return true;
    }
}

```

Ключевые отличия от InMemory

InMemory	EF Core
<code>_items.Add(entity)</code>	<code>_context.Entities.Add(entity) + SaveChanges()</code>
<code>_items.Find(x => x.Id == id)</code>	<code>_context.Entities.Find(id)</code>
Ручная генерация <code>Id</code>	EF Core автоматически присваивает <code>Id</code>
Фильтрация в памяти	Фильтрация преобразуется в SQL WHERE
Изменения сразу видны	Изменения применяются после <code>SaveChanges()</code>

Важные замечания

Не забывайте вызывать `SaveChanges()` :

- Без вызова `SaveChanges()` изменения не будут сохранены в БД
- EF Core отслеживает изменения, но не применяет их автоматически

Метод `CopyFrom` в Domain классах:

Для метода `Update` необходимо добавить метод `CopyFrom` в классы сущностей (в проекте `Domain`):

```

public void CopyFrom(<Сущность> other)
{
    // Копируем все свойства, КРОМЕ Id
    Свойство1 = other.Свойство1;
    Свойство2 = other.Свойство2;
    // ...
}

```

LINQ запросы выполняются лениво:

```

var query = _context.Entities.Where(x => x.Date > today); // Запрос НЕ выполнен

```

```
var list = query.ToList(); // Запрос выполнен ЗДЕСЬ
```

Замена InMemory на SqlServer в UI

Теперь необходимо изменить UI проект, чтобы он использовал реальную базу данных вместо хранения в памяти.

Шаг 1: Установка NuGet-пакетов конфигурации в UI проект

Для чтения файла `appsettings.database.json` в UI проекте установите следующие пакеты:

1. Щелкните правой кнопкой на проекте `<ИмяПроекта>.UI` → **Manage NuGet Packages**
2. Перейдите на вкладку **Browse**
3. Установите следующие пакеты (используйте ту же версию, что и EF Core пакеты):
 - `Microsoft.Extensions.Configuration`
 - `Microsoft.Extensions.Configuration.Json`

Важно: Версии должны соответствовать версии .NET проекта (например, для .NET 8 используйте версии 8.x).

Шаг 2: Копирование файла конфигурации в UI проект

// TODO Проверить, не стоит ли удалить эту главу

Файл `appsettings.database.json` нужен и в UI проекте для чтения строки подключения при запуске приложения.

Вариант 1: Скопировать файл

1. В Solution Explorer найдите файл `appsettings.database.json` в проекте `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`)
2. Скопируйте его (Ctrl+C)
3. Вставьте в проект `<ИмяПроекта>.UI` (Ctrl+V)
4. Щелкните правой кнопкой на новом файле → **Properties**
5. Установите **Copy to Output Directory** → **Copy if newer**

Вариант 2: Создать заново

1. В проекте `<ИмяПроекта>.UI` создайте файл `appsettings.database.json`
2. Скопируйте содержимое из файла в проекте `Data.SqlServer` (или `Data.MySQL`)
3. Установите **Copy to Output Directory** → **Copy if newer**

Примечание: Убедитесь, что строка подключения в обоих файлах одинаковая.

Шаг 3: Изменение App.xaml.cs

Необходимо переписать метод `OnStartup` в файле `App.xaml.cs` для использования EF Core вместо InMemory репозиториев.

Найдите файл: `<ИмяПроекта>.UI/App.xaml.cs`

Замените код на следующий:

```
using System.IO;
using System.Windows;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using <ИмяПроекта>.Data.Interfaces;
```

```

using <ИмяПроекта>.Data.SqlServer; // Или Data.MySQL для MySQL

namespace <ИмяПроекта>.UI;

public partial class App : Application
{
    private IReservationRepository _reservationRepository = null!;
    private IMasterRepository _masterRepository = null!;
    private <ИмяПроекта>DbContext _dbContext = null!;

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        // 1. Чтение конфигурации из файла
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.database.json")
            .Build();

        // 2. Создание DbContext через фабрику
        var factory = new <ИмяПроекта>DbContextFactory();
        _dbContext = factory.CreateDbContext(configuration);

        // 3. ВАЖНО: Применение миграций автоматически при запуске
        _dbContext.Database.Migrate();

        // 4. Создание репозиторий на основе DbContext
        _reservationRepository = new ReservationRepository(_dbContext);
        _masterRepository = new MasterRepository(_dbContext);

        // 5. Заполнение тестовыми данными (только если БД пустая)
        SeedInitData();

        // 6. Запуск главного окна
        var mainWindow = new ReservationsListWindow(_reservationRepository, _masterRepository);
        mainWindow.Show();
    }

    private void SeedInitData()
    {
        // Проверяем, есть ли уже данные в БД
        if (_masterRepository.GetAll().Any())
        {
            // Данные уже есть, пропускаем заполнение
            return;
        }

        // Остальной код SeedInitData остаётся без изменений
        // ...
    }

    protected override void OnExit(ExitEventArgs e)
    {
        // ВАЖНО: Освобождаем ресурсы DbContext при закрытии приложения
        _dbContext?.Dispose();
        base.OnExit(e);
    }
}

```

Конкретный пример для BeautyDB (SQL Server):

```

using System.IO;
using System.Windows;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using BeautyDB.Data.Interfaces;
using BeautyDB.Data.SqlServer;
using BeautyDB.Domain;

namespace BeautyDB.UI;

public partial class App : Application
{
    private IReservationRepository _reservationRepository = null!;
    private IMasterRepository _masterRepository = null!;
    private BeautyDbContext _dbContext = null!;

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        // 1. Чтение конфигурации
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.database.json")
            .Build();

        // 2. Создание DbContext
        var factory = new BeautyDbContextFactory();
        _dbContext = factory.CreateDbContext(configuration);

        // 3. Автоматическое применение миграций
        _dbContext.Database.Migrate();

        // 4. Создание репозиторий
        _reservationRepository = new ReservationRepository(_dbContext);
        _masterRepository = new MasterRepository(_dbContext);

        // 5. Заполнение тестовыми данными
        SeedInitData();

        // 6. Запуск главного окна
        var mainWindow = new ReservationsListWindow(_reservationRepository, _masterRepository);
        mainWindow.Show();
    }

    private void SeedInitData()
    {
        // Проверяем, есть ли уже мастера в БД
        if (_masterRepository.GetAll().Any())
        {
            return; // Данные уже есть
        }

        // Создание тестовых мастеров
        var master0Id = _masterRepository.Add(new Master { Name = "Анна Иванова" });
        var master0 = _masterRepository.Get(master0Id)!;
        var master1Id = _masterRepository.Add(new Master { Name = "Мария Петрова" });
        var master1 = _masterRepository.Get(master1Id)!;
        var master2Id = _masterRepository.Add(new Master { Name = "Елена Сидорова" });
        var master2 = _masterRepository.Get(master2Id)!;
    }
}

```

```
// Создание тестовых клиентов и резерваций
var client0 = new Client { Name = "Смирнова О.В.", Phone = "+7 916 234-5678" };

_reservationRepository.Add(new Reservation
{
    Id = Guid.NewGuid(),
    Client = client0,
    Master = master0,
    ServiceDescription = "Стрижка и укладка",
    Date = DateOnly.FromDateTime(DateTime.Today.AddDays(1)),
    StartTime = new TimeOnly(10, 0),
    Duration = TimeSpan.FromMinutes(60),
    Status = ReservationStatus.Created
});

// ... остальные тестовые данные
}

protected override void OnExit(ExitEventArgs e)
{
    _dbContext?.Dispose();
    base.OnExit(e);
}
}
```

Пояснение изменений

1. Чтение конфигурации:

```
var configuration = new ConfigurationBuilder()
    .SetBasePath(Directory.GetCurrentDirectory())
    .AddJsonFile("appsettings.database.json")
    .Build();
```

- Создаёт объект конфигурации, который читает строку подключения из файла `appsettings.database.json`
- `SetBasePath` указывает директорию, где находится файл
- `AddJsonFile` загружает JSON файл с настройками

2. Создание DbContext через фабрику:

```
var factory = new <ИмяПроекта>DbContextFactory();
_dbContext = factory.CreateDbContext(configuration);
```

- Использует фабрику, созданную ранее в проекте `Data.SqlServer` (или `Data.MySQL`)
- Передаёт конфигурацию в фабрику для получения строки подключения

3. Автоматическое применение миграций:

```
_dbContext.Database.Migrate();
```

- **Это ключевая строка!** Автоматически применяет все неприменённые миграции к базе данных
- При первом запуске создаст базу данных и все таблицы
- При последующих запусках применит только новые миграции (если они были добавлены)
- Избавляет от необходимости вручную запускать команду `dotnet ef database update`

Преимущества `Database.Migrate()` :

- Миграции применяются автоматически при каждом запуске

- Работает на любом компьютере (не нужно помнить про команду `database update`)
- Идеально подходит для учебных проектов и демонстраций

4. Создание репозиториев на основе DbContext:

```
_reservationRepository = new ReservationRepository(_dbContext);
_masterRepository = new MasterRepository(_dbContext);
```

- Вместо InMemory репозиториев создаём SqlServer (или MySQL) репозитории
- Передаём им `DbContext` для работы с базой данных

5. Проверка перед заполнением данными:

```
if (_masterRepository.GetAll().Any())
{
    return; // Данные уже есть
}
```

- **Важно:** Проверяем, есть ли данные в БД перед заполнением
- Без этой проверки при каждом запуске будут создаваться дубликаты записей
- InMemory хранилище очищалось при каждом запуске, но БД сохраняет данные навсегда

6. Освобождение ресурсов:

```
protected override void OnExit(ExitEventArgs e)
{
    _dbContext?.Dispose();
    base.OnExit(e);
}
```

- Вызываем `Dispose()` для закрытия соединения с БД при закрытии приложения
- Освобождает неуправляемые ресурсы (соединения, транзакции)

Шаг 4: Сборка и запуск приложения

1. Соберите решение: **Build** → **Build Solution** (`Ctrl+Shift+B`)
2. Убедитесь, что сборка прошла успешно без ошибок
3. Запустите приложение: **Debug** → **Start Debugging** (`F5`)

Что произойдёт при первом запуске:

1. Приложение прочтёт строку подключения из `appsettings.database.json`
2. EF Core подключится к серверу БД
3. Метод `Database.Migrate()` создаст базу данных (если её нет)
4. Будут созданы все таблицы согласно миграциям
5. Метод `SeedInitData()` заполнит БД тестовыми данными
6. Главное окно откроется и отобразит данные из БД

При последующих запусках:

1. EF Core подключится к существующей БД
 2. `Database.Migrate()` проверит и применит новые миграции (если есть)
 3. `SeedInitData()` обнаружит, что данные уже есть, и пропустит заполнение
 4. Главное окно отобразит сохранённые данные
-

Тестирование приложения и типичные проблемы

// TODO Прочитать это внимательно, проверить актуальность

Сценарии тестирования

После запуска приложения выполните следующие проверки для убеждения, что база данных работает корректно.

Тест 1: Проверка загрузки данных

1. Запустите приложение
2. Убедитесь, что главное окно открылось без ошибок
3. Проверьте, что в списке отображаются тестовые данные, созданные в `SeedInitData()`

Ожидаемый результат:

- Приложение открывается без исключений
- В списке видны записи (например, резервации для салона красоты)
- Данные корректно отображаются во всех столбцах

Тест 2: Создание новой записи

1. Нажмите кнопку **"Создать"** (или аналогичную в вашем приложении)
2. Заполните все поля в форме редактирования
3. Нажмите **"Сохранить"**
4. Убедитесь, что новая запись появилась в списке
5. **Закройте и перезапустите приложение**
6. Проверьте, что созданная запись всё ещё присутствует в списке

Ожидаемый результат:

- Новая запись успешно добавляется в список
- После перезапуска приложения запись не исчезла (данные сохранены в БД, а не в памяти)

Тест 3: Редактирование записи

1. Выберите любую запись в списке
2. Нажмите кнопку **"Редактировать"**
3. Измените несколько полей
4. Нажмите **"Сохранить"**
5. Убедитесь, что изменения отобразились в списке
6. **Закройте и перезапустите приложение**
7. Проверьте, что изменения сохранились

Ожидаемый результат:

- Изменения применяются к записи
- После перезапуска изменения не исчезли

Тест 4: Удаление записи

1. Выберите любую запись в списке
2. Нажмите кнопку **"Удалить"**
3. Подтвердите удаление в диалоговом окне
4. Убедитесь, что запись исчезла из списка
5. **Закройте и перезапустите приложение**

6. Проверьте, что запись не появилась снова

Ожидаемый результат:

- Запись удаляется из списка
- После перезапуска запись остаётся удалённой

Тест 5: Проверка данных в БД через GUI-инструмент

Этот тест подтверждает, что данные действительно сохраняются в базе данных.

Для SQL Server / LocalDB (через SSMS):

1. Откройте SQL Server Management Studio (SSMS)
2. Подключитесь к серверу:
 - Для LocalDB: (localdb)\MSSQLLocalDB
 - Для SQL Server Express: .\SQLEXPRESS
 - Для сервера колледжа: SQL-EDU (или другое имя сервера)
3. Разверните **Databases** → Найдите вашу БД (например, BeautyDB)
4. Разверните **Tables**
5. Щелкните правой кнопкой на таблице (например, dbo.Reservations) → **Select Top 1000 Rows**
6. Убедитесь, что данные из приложения видны в таблице

Для MySQL (через MySQL Workbench):

1. Откройте MySQL Workbench
2. Подключитесь к серверу
3. Выберите вашу схему (БД) в списке слева
4. Разверните **Tables**
5. Щелкните правой кнопкой на таблице → **Select Rows - Limit 1000**
6. Убедитесь, что данные из приложения видны в таблице

Ожидаемый результат:

- Таблицы созданы в базе данных
- Записи, созданные через приложение, видны в таблицах
- Изменения и удаления отражены в БД

Типичные проблемы и их решения

Проблема 1: FileNotFoundException для appsettings.database.json

Ошибка:

```
System.IO.FileNotFoundException: Could not find file 'C:\...\appsettings.database.json'
```

Причина:

Файл конфигурации не копируется в директорию сборки (bin\Debug\net9.0\).

Решение:

1. В Solution Explorer найдите файл appsettings.database.json в проекте <ИмяПроекта>.UI
2. Щелкните правой кнопкой → **Properties**
3. Проверьте, что **Copy to Output Directory** установлено в **Copy if newer** (или **Copy always**)
4. Пересоберите проект (Ctrl+Shift+B)
5. Проверьте, что файл появился в папке bin\Debug\net9.0\ (или bin\Debug\net8.0\)

Проблема 2: Ошибка подключения к SQL Server LocalDB

Ошибка:

```
Microsoft.Data.SqlClient.SqlException: A network-related or instance-specific error occurred while establishing a connection to SQL Server.
```

Причина:

SQL Server LocalDB не запущен или не установлен.

Решение:

Шаг 1: Проверить наличие LocalDB

```
sqllocaldb info
```

Если команда не найдена — LocalDB не установлен.

Шаг 2: Установить LocalDB через Visual Studio Installer

1. Откройте Visual Studio Installer
2. Нажмите **Modify** для вашей версии Visual Studio
3. Перейдите на вкладку **Individual components**
4. Найдите и отметьте **SQL Server Express LocalDB**
5. Нажмите **Modify** и дождитесь установки

Шаг 3: Запустить LocalDB

```
sqllocaldb start MSSQLLocalDB  
sqllocaldb info MSSQLLocalDB
```

Шаг 4: Проверить строку подключения

Убедитесь, что в `appsettings.database.json` указана правильная строка:

```
{  
  "ConnectionStrings": {  
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=<ИмяБД>;Integrated  
Security=True;TrustServerCertificate=True;"  
  }  
}
```

Проблема 3: Ошибка подключения к SQL Server Express

Ошибка:

```
Microsoft.Data.SqlClient.SqlException: Cannot open database requested by the login.
```

Решение:

Шаг 1: Проверить, что SQL Server Express запущен

1. Откройте **Services** (Win+R → `services.msc`)
2. Найдите службу **SQL Server (SQLEXPRESS)**
3. Убедитесь, что статус **Running**
4. Если остановлена — щелкните правой кнопкой → **Start**

Шаг 2: Проверить строку подключения

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost\\SQLEXPRESS;Database=
<ИмяБД>;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

Шаг 3: Проверить доступ через SSMS

1. Откройте SQL Server Management Studio
2. Попробуйте подключиться к localhost\SQLEXPRESS с **Windows Authentication**
3. Если подключение не удаётся — проверьте конфигурацию SQL Server

Проблема 4: Ошибка подключения к MySQL

Ошибка:

```
MySqlConnection.MySqlException: Unable to connect to any of the specified MySQL hosts.
```

Решение:

Шаг 1: Проверить, что MySQL запущен

Windows:

1. Откройте **Services** (Win+R → `services.msc`)
2. Найдите службу **MySQL80** (или другая версия)
3. Убедитесь, что статус **Running**
4. Если остановлена — щелкните правой кнопкой → **Start**

Linux / macOS:

```
sudo systemctl status mysql
sudo systemctl start mysql
```

Шаг 2: Проверить логин и пароль

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=<ИмяБД>;User=root;Password=<ВашПароль>;"
  }
}
```

Шаг 3: Проверить доступ через MySQL Workbench

1. Откройте MySQL Workbench
2. Попробуйте подключиться с теми же учётными данными
3. Если подключение не удаётся — проверьте пароль или создайте нового пользователя

Проблема 5: Таблицы не создаются автоматически

Симптомы:

- Приложение запускается без ошибок
- Но данных нет, или возникает ошибка при попытке добавить запись

Причина:

Миграции не применились автоматически.

Решение:

Шаг 1: Проверить наличие строки `Database.Migrate()`

Откройте `App.xaml.cs` и убедитесь, что в методе `OnStartup` присутствует:

```
_dbContext.Database.Migrate();
```

Шаг 2: Применить миграции вручную

Если проблема сохраняется, выполните команду вручную:

```
dotnet ef database update --project <ИмяПроекта>\<ИмяПроекта>.Data.SqlServer
```

Шаг 3: Проверить, что миграция создана

Убедитесь, что в проекте `<ИмяПроекта>.Data.SqlServer` (или `Data.MySQL`) существует папка `Migrations/` с файлами миграций.

Если папки нет — создайте миграцию:

```
dotnet ef migrations add InitialCreate --project <ИмяПроекта>\<ИмяПроекта>.Data.SqlServer
```

Проблема 6: Дубликаты тестовых данных при каждом запуске

Симптомы:

- При каждом запуске приложения в БД добавляются новые тестовые записи
- Через несколько запусков накапливается множество одинаковых записей

Причина:

Отсутствует проверка в методе `SeedInitData()`.

Решение:

Добавьте проверку в начало метода `SeedInitData()` в файле `App.xaml.cs`:

```
private void SeedInitData()
{
    // Проверяем, есть ли уже данные в БД
    if (_masterRepository.GetAll().Any())
    {
        return; // Данные уже есть, пропускаем заполнение
    }

    // Остальной код создания тестовых данных...
}
```

Или проверьте конкретную сущность:

```
if (_reservationRepository.GetAll(ReservationsFilter.Empty).Any())
{
    return;
}
```

Проблема 7: `InvalidOperationException` при работе со связанными сущностями

Ошибка:

```
System.InvalidOperationException: The instance of entity type 'Master' cannot be tracked because another instance with the same key value is already being tracked.
```

Причина:

EF Core пытается отслеживать несколько экземпляров одной и той же сущности.

Решение:

Используйте метод `AsNoTracking()` при чтении данных, если они не будут изменяться:

```
public List<Reservation> GetAll(ReservationsFilter filter)
{
    var query = _context.Reservations
        .Include(r => r.Master) // Загружаем связанные данные
        .AsNoTracking()         // Отключаем отслеживание
        .AsQueryable();

    // Фильтрация...

    return query.ToList();
}
```

Или используйте `Attach()` для присоединения существующих сущностей:

```
public Guid Add(Reservation reservation)
{
    if (reservation.Master != null)
    {
        // Присоединяем существующего мастера, чтобы EF Core не пытался создать нового
        _context.Attach(reservation.Master);
    }

    _context.Reservations.Add(reservation);
    _context.SaveChanges();
    return reservation.Id;
}
```

Проблема 8: Ошибка сериализации при работе с `DateOnly` или `TimeOnly`

Ошибка в старых версиях EF Core:

```
System.InvalidOperationException: The property 'Date' could not be mapped because it is of type 'DateOnly'.
```

Причина:

Используется старая версия EF Core (ниже 6.0), которая не поддерживает `DateOnly` и `TimeOnly`.

Решение:

Вариант 1: Обновить EF Core до версии 6.0 или выше

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.11
```

Вариант 2: Использовать `DateTime` вместо `DateOnly/TimeOnly`

Измените свойства в классах Domain:

```
public class Reservation
{
    public DateTime Date { get; set; } // Вместо DateOnly
    public TimeSpan StartTime { get; set; } // Вместо TimeOnly
    // ...
}
```

Проверка логов миграций

Чтобы убедиться, что миграции применились, проверьте таблицу `__EFMigrationsHistory` в базе данных.

Через SSMS (SQL Server):

```
SELECT * FROM __EFMigrationsHistory;
```

Через MySQL Workbench:

```
SELECT * FROM `__EFMigrationsHistory`;
```

Ожидаемый результат:

Таблица содержит строки с именами применённых миграций (например, `20250112_InitialCreate`).

Если таблица пустая — миграции не применились. Вернитесь к Проблеме 5.

Ожидаемый результат

По итогам работы должно быть реализовано:

- **Архитектура:** Новый проект `<ИмяПроекта>.Data.SqlServer` (или `<ИмяПроекта>.Data.MySQL`) с корректными ссылками на `Domain` и `Data.Interfaces`, а также ссылка из `UI` на новый проект
- **DbContext:** Класс `<ИмяПроекта>DbContext` со свойствами `DbSet<T>` для всех сущностей приложения
- **Конфигурация:** Файл `appsettings.database.json` с корректной строкой подключения, скопированный в оба проекта (`Data.SqlServer` и `UI`) с настройкой копирования в выходную директорию
- **Фабрика:** Класс `<ИмяПроекта>DbContextFactory`, реализующий `IDesignTimeDbContextFactory`, для создания контекста как при миграциях, так и в `UI`
- **Миграции:** Папка `Migrations/` с первой миграцией `InitialCreate`, содержащая код для создания всех таблиц
- **Репозитории EF Core:** Реализация всех интерфейсов из `Data.Interfaces` с использованием `EF Core` вместо `List<T>`
- **Обновлённый UI:** Изменённый файл `App.xaml.cs`:
 - Чтением конфигурации из `JSON`
 - Созданием `DbContext` через фабрику
 - Автоматическим применением миграций (`Database.Migrate()`)
 - Созданием репозитория на основе `DbContext`
 - Проверкой данных перед заполнением тестовыми записями
 - Освобождением ресурсов при закрытии приложения
- **База данных:** Созданная БД с таблицами для всех сущностей и служебной таблицей `__EFMigrationsHistory`
- **Функциональность:** Все CRUD операции работают через базу данных, данные сохраняются между запусками приложения
- **Тестирование:** Проверенная работа создания, редактирования, удаления записей с перезапуском приложения

Ожидаемая структура проекта

```
Solution/
├─ <ИмяПроекта>.UI/                # WPF формы и элементы управления
├─ <ИмяПроекта>.Domain/            # Классы предметной области
├─ <ИмяПроекта>.Services/          # Бизнес-логика и сервисы
├─ <ИмяПроекта>.Data.Interfaces/    # Интерфейсы репозитория
├─ <ИмяПроекта>.Data.InMemory/      # In-memory реализация (опционально)
└─ <ИмяПроекта>.Data.SqlServer/    # EF Core для SQL Server
    ├─ <ИмяПроекта>DbContext.cs
    ├─ <ИмяПроекта>DbContextFactory.cs
    ├─ <Сущность>Repository.cs (для каждой сущности)
    ├─ appsettings.database.json
    └─ Migrations/
        ├─ <Timestamp>_InitialCreate.cs
        ├─ <Timestamp>_InitialCreate.Designer.cs
        └─ <ИмяПроекта>DbContextModelSnapshot.cs
```

Полезные ссылки

- Документация Entity Framework Core: <https://learn.microsoft.com/ef/core/>
- Миграции в EF Core: <https://learn.microsoft.com/ef/core/managing-schemas/migrations/>
- SQL Server Management Studio (SSMS): <https://learn.microsoft.com/sql/ssms/download-sql-server-management-studio-ssms>
- MySQL Workbench: <https://dev.mysql.com/downloads/workbench/>